

# 程序设计实践

## The Practice of Programming

### 排错(Debugging)

# 什么是BUG，什么是DEBUG

**bug.**

**b.** A defect or fault in a machine, plan, or the like. orig. *U.S.*

1889 Pall Mall Gaz. 11 Mar. 1/1 Mr. Edison, I was informed, had been up the two previous nights discovering 'a bug' in his phonograph—an expression for solving a difficulty, and implying that some imaginary insect has secreted itself inside and is causing all the trouble.

*Oxford English Dictionary. 2nd Edition*

DEBUG，比“排错”更加生动的另一个翻译：“逮BUG”

9/9

13 MC (032) MP - MC ~~2.130476415 (-2)~~ 4.615925059(-2)  
(033) PRO 2 2.130476415  
conv 2.130676415

Relay 214 E  
Relay 3375

1100 Started Cosine Tap (Sine check)  
1525 Started Mult + Adder Test.

1545

Relay #70 Panel F  
(moth) in relay.

First actual case of bug being found.  
1630 antenae started.  
1700 closed down.

1947年9月9日，Grace Hopper在Mark II 计算机里发现了一个真正的“BUG”，居然用胶条将这个“BUG”粘贴在报告里。

显然，bug和debug这样的用法，应该是在这之前就有，她这次终于抓到了一只真正的bug，于是在报告里这样写“First actual case of bug being found”

# 如何对待BUG

## 首先，最好不要出BUG

采用良好的设计避免BUG。

采用良好的编程风格，编程习惯来避免BUG。

注意所采用的语言的编译器的告警，不要总是忽略它们。

## 其次，让BUG容易发现

设计良好的用户界面（日志，错误信息，查看命令）

边界条件的检查以及断言（assert）的使用

## 再次，即使出现BUG，也要自己先发现并解决

完善的单元测试用例

设计自动测试脚本

## 再再次，自己没有发现，也要尽可能让组织内部的测试人员发现

组织内要有完善的测试制度

## 最后，不可避免，发布给用户的程序还是会出BUG...

# 如何对待BUG

一般应用程序运行出现**BUG**，首先，要99.99%确信：

- 不会是硬件的问题导致
- 不会是网络传输的错误导致
- 不会是操作系统的**BUG**导致
- 不会是编译器的**BUG**导致
- 不会是标准库的**BUG**导致

程序运行出现**BUG**，不知为何，**BUG**又消失了。但是实际上：

- **BUG**肯定仍然存在
- 而且这可能是一个比较难找的**BUG**
- 应该尽可能让这个**BUG**复现，以便解决它。

端正态度，努力从自己的代码找问题，尽早将**BUG**彻底解决掉。

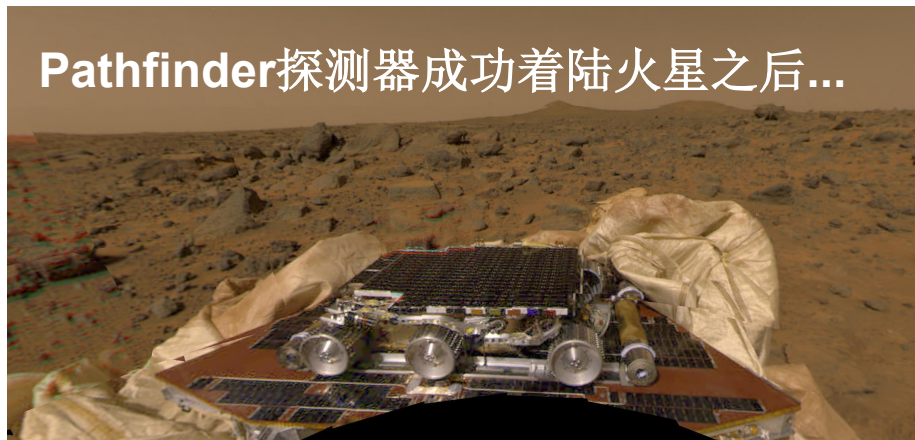
**注意：BUG越晚解决，解决它所需成本越高**



# BUG越晚解决，解决它所需成本越高

这是个发生在1997年的  
真实案例

Pathfinder探测器成功着陆火星之后...

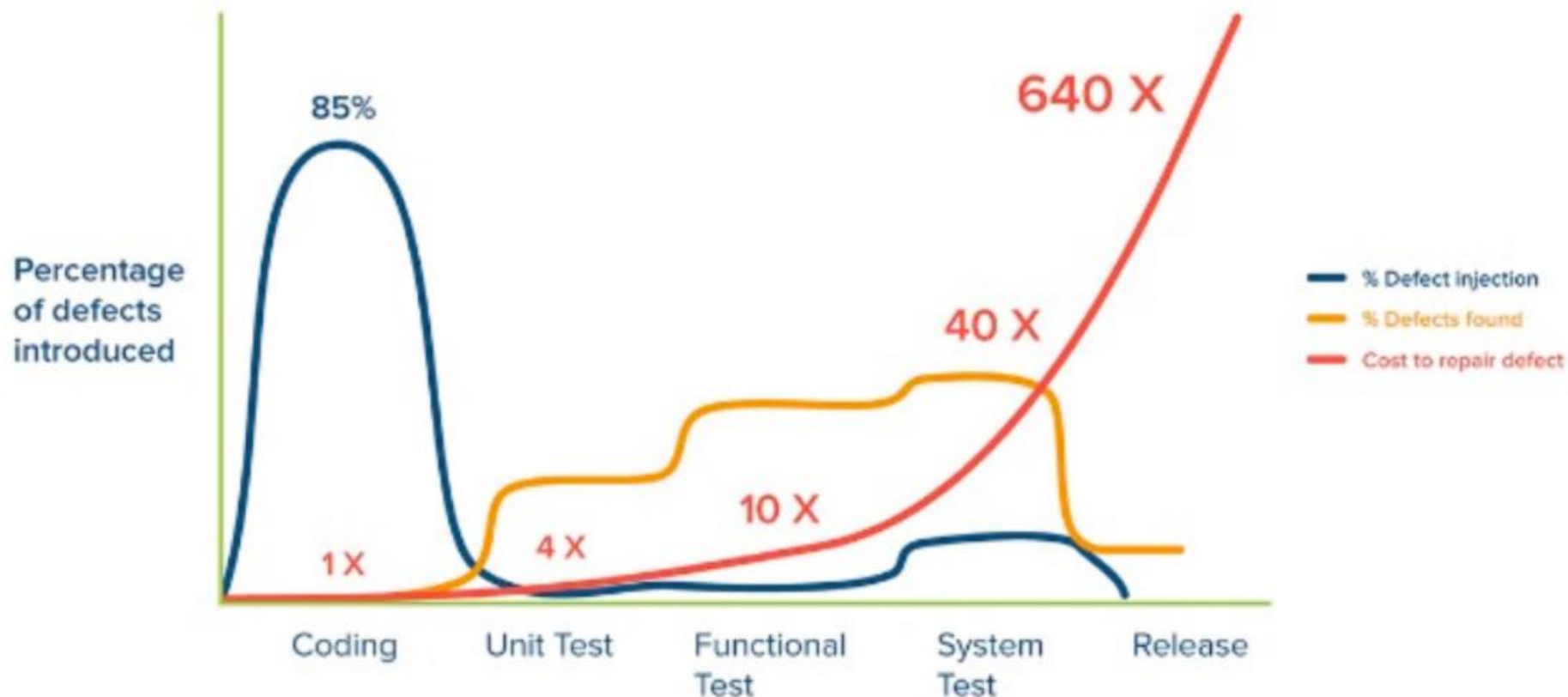


他们发现了  
Pathfinder的程序  
有一个BUG...



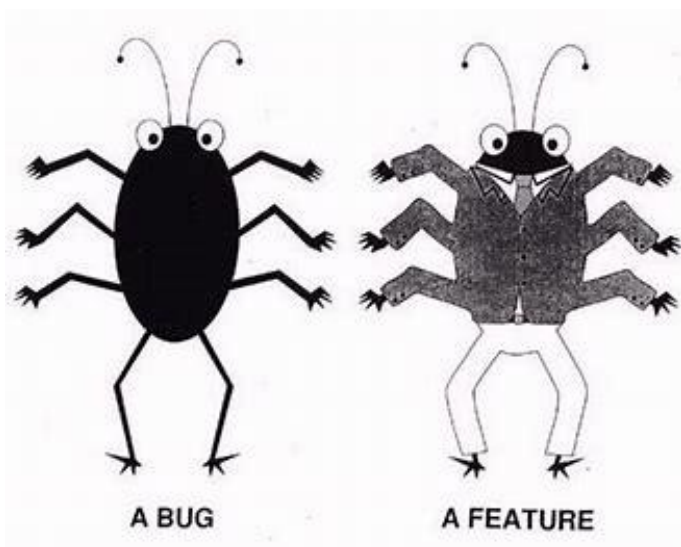
实际上这个BUG在地面测试中出现过，  
但是没有引起开发人员的注意。

# BUG越晚解决，解决它所需成本越高



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

# BUG的归宿 - 除了被发现并解决掉之外...



少数BUG会因为用户的喜爱变成FEATURE。

Minecraft 中特性梗来自于 Dinnerbone。  
Dinnerbone: 为漏洞变为新的特性而欢呼! 我在制作附魔台的时候出现了一个 bug——附魔台突然发光了, 但是我觉得还蛮酷的, 所以就改成特性了



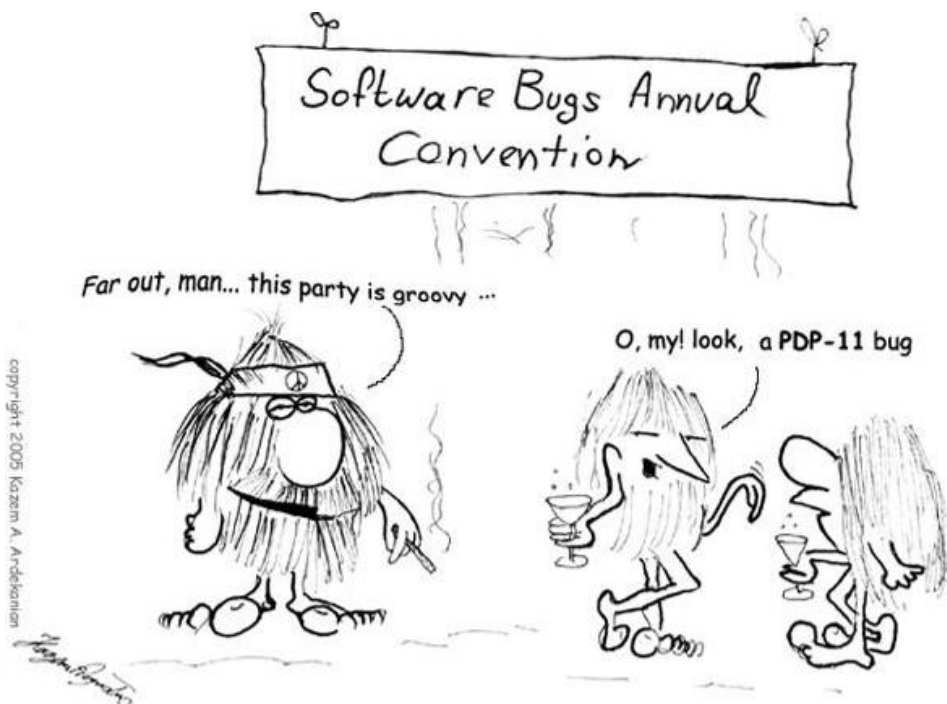


# BUG的归宿 - 除了被发现并解决掉之外...

也有少数BUG会因为要保持兼容不得不变成FEATURE。

一个接口消息返回的错误码写错了，但是这个接口的大量调用者已经这么用了，只好将错就错。

有的BUG会一直留在程序中成为老BUG。



# DEBUG工具(debugger)

大部分语言的IDE都集成有**debugger**，可以对程序执行单步跟踪，可以实时查看所有变量，但是，它们在处理一些较难的**BUG**的时候用途有限。

**linux**或者**unix**下都有基于命令行的调试工具**gdb**或者**dbx**，它功能上覆盖了IDE支持的所有功能，例如设置断点，单步执行，查看变量等。

**gdb**或者**dbx**经常用于查看程序崩溃时留下的**core**文件，可以方便地查看崩溃时的函数调用栈状态，指出崩溃在当前栈顶函数的哪一行。但是，如果指针的误用导致栈被严重破坏，这时候此工具就没有作用了。

**java**程序执行遇到异常的时候，一般可以输出当前的堆栈信息有助于**BUG**的分析。

不过，工具解决的一般都是比较显而易见的**BUG**。很多难以解决的**BUG**，都是依靠在程序中增加类似**printf**等打印语句的方法一点一点逻辑推理解决的。

# DEBUG工具(debugger) - vscode

The screenshot displays the Visual Studio Code (VS Code) interface during a C++ debugging session using GDB. The interface is divided into several panels:

- Left Sidebar:** Contains icons for Explorer, Search, Run and Debug, and Extensions. The 'Run and Debug' icon is active.
- Top Panel:** Shows the '调试' (Debug) tab with 'C++ Launch (GDB)' selected. The file '1.cpp' is open in the editor.
- Editor:** Displays the source code of '1.cpp'. The code is as follows:

```
1 #include <stdio.h>
2 int main(int argc, char *args[]){
3     int i, k;
4     printf("hello, world!\n");
5     printf("argc:%d\nargv:\n",argc);
6     for(i = 0; i < argc; i++){
7         printf("%d : %s\n", i, args[i]);
8     }
9     getchar();
10    return 0;
11 }
```

The line `printf("argc:%d\nargv:\n",argc);` is highlighted, indicating the current execution point.
- Bottom Panel:** Contains the '输出' (Output) window, which shows the program's output: `hello, world!`.
- Variables Panel (Locals):** Located on the left, it shows the current state of variables:
  - `i: 4194432 [int]`
  - `argc: 4 [int]`
  - `args: 0x690db8 [char **]`
- Call Stack (调用堆栈):** Located on the left, it shows the current function call stack:
  - `main(int argc, char ** args) 1.cpp 5`
- Breakpoints (断点):** Located on the left, it shows the current breakpoints:
  - `1.cpp 5`
  - `1.cpp 7`

# DEBUG工具(debugger) - gdb

```
1 #include <stdio.h>
2 int main(int argc, const char* args[]){
3     int i, k;
4     printf("hello, world!\n");
5     printf("argc:%d\nargv:\n", argc);
6     for(i = 0; i < argc; i++){
7         printf("%d : %s\n", i, args[i]);
8     }
9     getchar();
10    return 0;
11 }
```

```
$ g++ -o gdbtest -g gdbtest.c
```

编译之后, 使用gdb 可执行文件名 开始debug

例如可以使用这些命令:

b 6 // 在第6行设置断点

r 111 222 333 // 运行程序, 其中111 222 333是运行程序给出的命令行参数

p i // 查看变量i的值

p argc // 查看变量argc的值

```
$ gdb gdbtest
```

```
...
```

```
(gdb) b 6
```

```
Breakpoint 1 at 0x400671: file gdbtest.c, line 6.
```

```
(gdb) r 111 222 333
```

```
Starting program: /home/hbzhang/test/gdbtest 111 222 333
```

```
hello, world!
```

```
argc:4
```

```
argv:
```

```
Breakpoint 1, main (argc=4, argv=0x7fffffffe258) at gdbtest.c:6
```

```
6         for(i = 0; i < argc; i++){
```

```
(gdb) p i
```

```
$1 = 0
```

```
(gdb) p argc
```

```
$2 = 4
```

```
(gdb)
```

# DEBUG工具(debugger) - gdb

```
1 #include <stdio.h>
2 int inputInt()
3 {
4     int i;
5     scanf("%d", i);
6     return i;
7 }
8 int main()
9 {
10     int a = inputInt();
11     printf("%d\n", a);
12     return 0;
13 }
```

```
$ g++ -g gdb.C
```

```
$ ./a.out
123
段错误 (core dumped)
```

```
$ ls core.*
core.44508
```

```
$ gdb a.out core.44508
GNU gdb (GDB) Red Hat Enterprise Linux (7.2-75.el6)
...
(gdb) where
#0  0x000000357ae56fbb in _IO_vfscanf_internal (s=<value optimized out>,
format=0x40074a "", argptr=0x7fffffa355630, errp=0x0) at vfscanf.c:1826
#1  0x000000357ae65cdc in __scanf (format=<value optimized out>) at scanf.c:33
#2  0x00000000000400620 in inputInt () at gdb.C:5
#3  0x00000000000400632 in main () at gdb.C:10
(gdb)
```

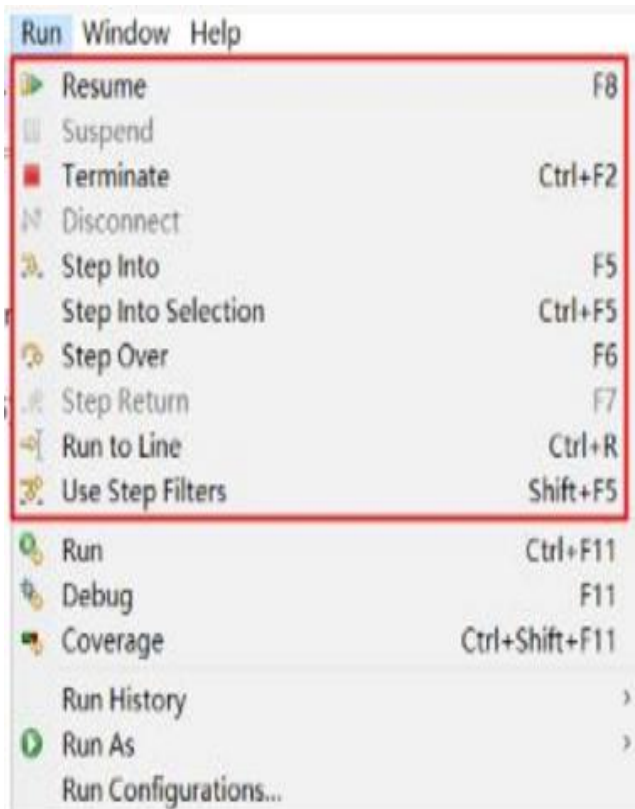


# DEBUG工具 - java的printStackTrace

```
1 import java.io.*;
2 public class Hello {
3     public static int div(int a, int b) throws Exception{
4         return a / b;
5     }
6     public static void main(String args[]){
7         try{
8             int i = div(100, 0);
9         }
10        catch(Exception e){
11            e.printStackTrace();
12        }
13    }
14 }
```

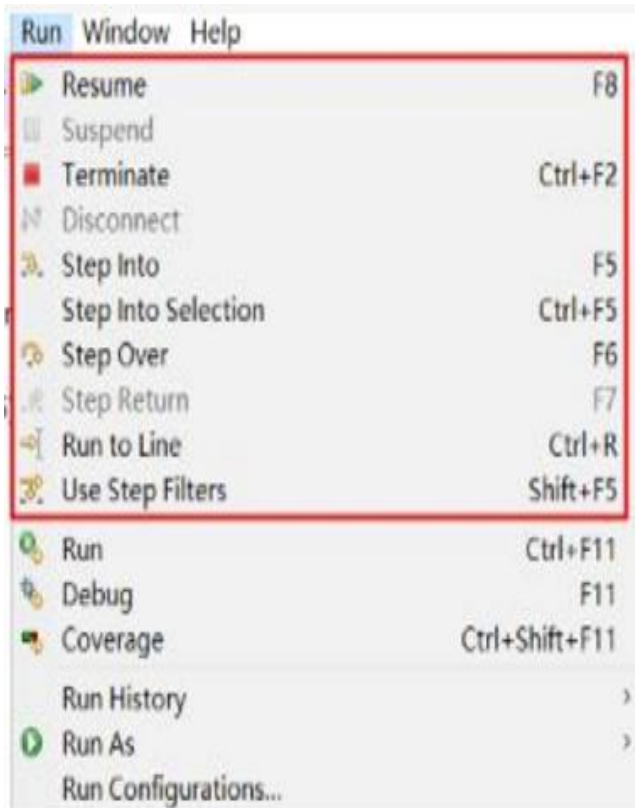
```
$ javac Hello.java
$
$
$ java Hello
java.lang.ArithmeticException: / by zero
    at Hello.div(Hello.java:4)
    at Hello.main(Hello.java:8)
```

# DEBUG工具 - java Eclipse中的常用debug命令-1



| 调试指令            | 快捷方式     | 说明                                   |
|-----------------|----------|--------------------------------------|
| Debug           | F11      | 开启调试                                 |
| Terminate       | Ctrl+F2  | 终止调试                                 |
| Step Into       | F5       | 跳入，（下一步，进入所调用的函数）                    |
| Step Over       | F6       | 跳过，（下一步，不进入所调用的函数）                   |
| Step Return     | F7       | 跳出，（执行完当前函数，然后return跳出该函数）           |
| Use Step Filter | Shift+F5 | 逐步过滤，一直执行直到遇到未经过滤的位置或断点（之前已经设置过过滤条件） |
| Resume          | F8       | 重新开始执行debug，一直运行直到遇到断点               |

# DEBUG工具 - java Eclipse中的常用debug命令-2



| 调试指令                 | 快捷方式 | 说明                                            |
|----------------------|------|-----------------------------------------------|
| hit count            |      | 设置执行次数，适合程序中的（设置 breakpoint view-右键hit count） |
| inspect              |      | 检查、运算。执行一个表达式显示执行值。                           |
| watch                |      | 实时地监视变量的变化                                    |
| Breakpoint           |      | line breakpoint, 行断点, 在该行暂停                   |
| Field Breakpoint     |      | 也叫watchpoint(监视点), 当成员变量被读取或修改时暂停             |
| Method Breakpoint    |      | 进入/离开此方法时暂停                                   |
| Exception Breakpoint |      | 捕抓到Exception时暂停                               |

# DEBUG的一般步骤

1. 了解清楚BUG的表现
2. 获取这个BUG的线索
3. 查阅相关版本的代码修改
4. 在自己的开发环境复现BUG
5. 在复现BUG的开发环境下分析
6. 二分法缩小BUG的范围
7. 不能复现的(Non-reproducible) BUG的处理

# 1. 了解清楚BUG的表现

**BUG的表现，包括如下几个方面：**

出现BUG的前置条件（主机、操作系统、程序版本、配置等等）

出现BUG时的输入或操作

出现BUG之后的现象（输出不正确？死循环？异常退出？）

**BUG发现的时机，影响对BUG表现的了解**

如果是自己发现的错误，自己直接可以了解。

如果是组织内测试人员或运维人员发现的BUG，也比较容易了解到BUG的表现，但不如前者。

如果是用户投诉，了解BUG表现则相对困难。

所以，我们应该让BUG尽早被发现。

**了解这个BUG表现的普遍性和特殊性**

以前是否出现过类似的BUG？

主机，操作系统，版本这些条件是否特殊？

是偶然发生？还是频繁发生？还是必然发生？



## 2. 获取这个BUG的线索

### BUG发生前后这段时间的日志

基于良好的接口设计，理应包含完善的日志文件，但是：

也许怀疑有问题的地方并没有记录日志（后悔中，为什么这里当时不加个日志呢？）

BUG发生时，也许有用的日志开关并未打开

也许日志太多，报告BUG的时候已经被冲掉

### UNIX或者Linux系统下，如果发生异常退出，获取core文件

理想情况下，使用dbx或者gdb查看core文件，可以定位出问题的文件以及行号，但是：

某些情况下退出，并不会产生core文件

也许用户设置了禁止core文件产生

有一些指针越界导致的异常，堆栈被破坏得很严重，core文件无法提供有用的信息

### 使用维护命令获取尽可能多的信息

基于良好的接口设计，理应有命令可以查看当前的程序状态，但是：

严重的BUG导致程序退出，则现场就会被破坏，所有信息都已经被初始化。

BUG报告时间太晚，当前的状态无法提供有用的信息。

也许你关心的地方，并没有对应的维护命令（后悔中，为什么这里当时不加一个命令呢？）

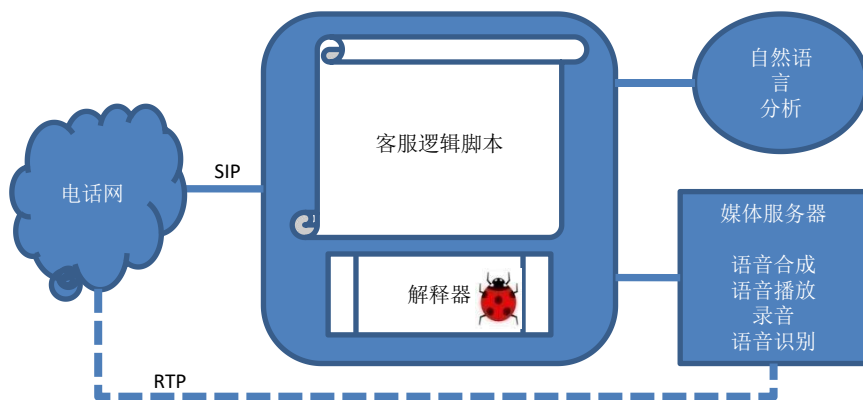
## 3. 查阅相关版本的代码修改

基于良好的习惯，所有代码的所有版本都应被版本管理工具妥善的管理

如果版本n出现的BUG，可以diff版本n与版本n-1的差别，仔细阅读这些差别

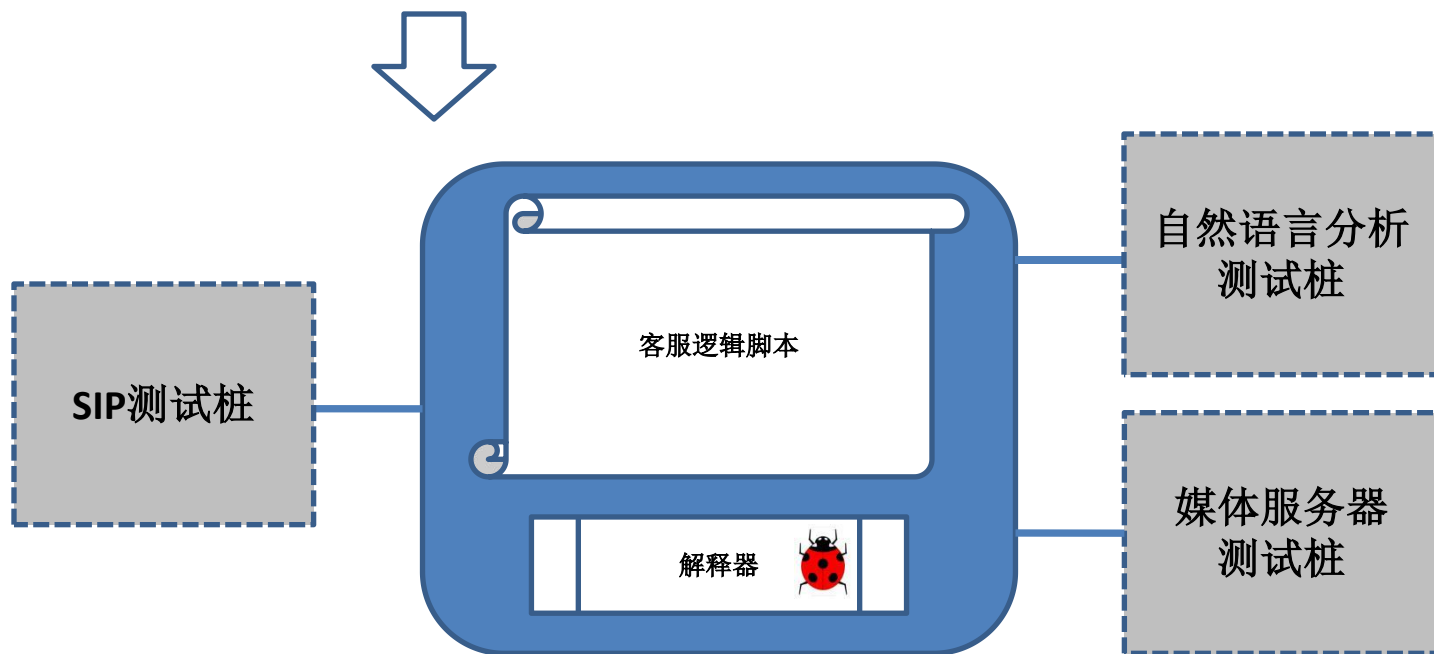


## 4. 在自己的开发环境复现BUG



开发环境往往无法与实际运行环境一致。

需要设计各种测试桩尽可能模拟实际运行环境。



## 5. 在复现BUG的开发环境下分析

可以在有所怀疑的地方，任意增加日志，增加断言，增加命令。

尽可能认真地增加这些东西，让它们以后还有用。



天哪！我加上一行日志输出，**BUG**就不见了！  
去掉日志**BUG**又出来了！

上面的问题不要奇怪，因为指针导致的问题，任何不可思议的现象都可能出现。

既然**BUG**消失了，那么刚才这个环境就是一个不能复现**BUG**的环境，毫无用处。

恢复到以前，重新修改。

## 6. 二分法缩小BUG的范围

(1-100行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(1-50行脚本 + 解释器) 执行10000次 => 没有内存泄漏

(51-100行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(51-75行脚本 + 解释器) 执行10000次 => 发现内存泄漏

(51-63行脚本 + 解释器) 执行10000次 => 没有内存泄漏

(64-75行脚本 + 解释器) 执行10000次 => 发现内存泄漏

范围缩小之后，只有12行脚本，就有内存泄露，那么问题必然在这12行脚本涉及到的代码中。

# 7. 不能复现的(Non-reproducible) BUG

确实有些BUG难以复现，或者需要很大的成本才能复现。尝试做这些事情：

1、使用工具扫描代码（例如splint），对工具给出的可疑结果认真检查。

2、回顾自己的经验教训，例如：

- 检查是否有没有初始化的变量？是否有需要返回值的函数没有返回值？

- 检查是否有错误的指针应用？

  - 函数返回了栈里的指针？

  - 指针重复删除？

  - 使用了删除指针指向的数据？

- 程序是否涉及多线程？

  - 是否你使用的库启动了多线程你不知道？

  - 是否有未加锁的共享资源访问？

  - 是否使用了线程不安全的函数？

3、把你的BUG，你的代码讲给你周围的人听。





```
for (p = head; p != NULL; p = p->next)
    free(p);
```

```
for (p = head; p != NULL; )
{
    next = p->next;
    free(p);
    p = next;
}
```

已经释放的内存，不应该再次使用它。

```
char* msg(int n, const char* s)
{
    char buf[100];
    sprintf(buf, "error: %d: %s", n, s);
    return buf;
}
```

```
char* msg(int n, const char* s)
{
    static char buf[100];
    sprintf(buf, "error: %d: %s", n, s);
    return buf;
}
```

函数的局部变量，在函数退出之后就失效了。

```
memset(p, n, 0); // 将n个0存储在p中
```

```
memset(p, 0, n); // 将n个0存储在p中
```

`void *memset(void *str, int c, size_t n)` 复制字符 `c`（一个无符号字符）到参数 `str` 所指向的字符串的前 `n` 个字符。

检查一下参数的次序是否用错了？

有时候，参数的次序并不是那么理所当然，而且编译不会出现问题。

```
void outputErr(int errCode, const char* errInfo)
{
    printf("ERROR: [%d]%s", errInfo, errCode);
}
```

```
void outputErr(int errCode, const char* errInfo)
{
    printf("ERROR: [%d]%s", errCode, errInfo);
}
```

检查一下，printf给出的格式串和具体的参数类型是否匹配？



```
int n;  
scanf("%d", n);
```

```
int n;  
scanf("%d", &n);
```

检查一下，scanf给出的接收数据的参数，是不是有忘记使用地址的情况？

```
void printHex(const char* s, int length)
{
    for(int i = 0; i <= length; i++)
        printf("%02X ", (unsigned char) (s[i]));
}
```

```
void printHex(const char* s, int length)
{
    for(int i = 0; i < length; i++)
        printf("%02X ", (unsigned char) (s[i]));
}
```

检查一下，数组访问的边界条件是否弄错？

```
char buffer[MAX_BUF_LEN];  
if(length <= MAX_BUF_LEN);  
{  
    memcpy(buffer, message, length);  
}
```

```
char buffer[MAX_BUF_LEN];  
if(length <= MAX_BUF_LEN)  
{  
    memcpy(buffer, message, length);  
}
```

检查一下，if的条件之后有没有不小心加了个分号？



```
bool sendMessage(const char* msg, int length)
{
    if(...)
        return false;
    if(...)
        return false;
    sendToWire(msg, length);
}
```

```
bool sendMessage(const char* msg, int length)
{
    if(...)
        return false;
    if(...)
        return false;
    return sendToWire(msg, length);
}
```

检查一下定义了返回值的函数，是否真的每个分支都确实有返回值。

```
const char* getCurrentTime() // 返回HH:MM:SS格式的当前时间字符串
{
    static char timeStr[256];
    time_t t = time(NULL);
    struct tm* tm = localtime(&t);
    strftime(timeStr, sizeof(timeStr), "%T", tm);
    return timeStr;
}
```

```
const char* t1 = getCurrentTime();
// ...这里有个耗费较长时间的功能
const char* t2 = getCurrentTime();
printf("from %s to %s\n", t1, t2);
```

```
std::string t1 = getCurrentTime();
// ...这里有个耗费较长时间的功能
std::string t2 = getCurrentTime();
printf("from %s to %s\n", t1.c_str(), t2.c_str());
```

线程不安全函数，往往返回了相同的**static**存储空间的地址，后面的调用会覆盖前面的调用。

|                    |                 |                 |                    |
|--------------------|-----------------|-----------------|--------------------|
| asctime()          | basename()      | catgets()       | crypt()            |
| ctime()            | dbm_clearerr()  | dbm_close()     | dbm_delete()       |
| dbm_error()        | dbm_fetch()     | dbm_firstkey()  | dbm_nextkey()      |
| dbm_open()         | dbm_store()     | dirname()       | dLError()          |
| drand48()          | encrypt()       | endgrent()      | endpwent()         |
| endutxent()        | ftw()           | getc_unlocked() | getchar_unlocked() |
| getdate()          | getenv()        | getgrent()      | getgrgid()         |
| getgrnam()         | gethostent()    | getlogin()      | getnetbyaddr()     |
| getnetbyname()     | getnetent()     | getopt()        | getprotobyname()   |
| getprotoynumber()  | getprotoent()   | getpwent()      | getpwnam()         |
| getpwuid()         | getservbyname() | getservbyport() | getservent()       |
| getutxent()        | getutxid()      | getutxline()    | gmtime()           |
| hcreate()          | hdestroy()      | hsearch()       | inet_ntoa()        |
| l64a()             | lgamma()        | lgammaf()       | lgammal()          |
| localeconv()       | localtime()     | lrand48()       | mrnd48()           |
| nftw()             | nl_langinfo()   | ptsname()       | putc_unlocked()    |
| putchar_unlocked() | putenv()        | pututxline()    | rand()             |
| readdir()          | setenv()        | setgrent()      | setkey()           |
| setpwent()         | setutxent()     | strerror()      | strtok()           |
| ttyname()          | unsetenv()      | wcstombs()      | wctomb()           |

上面列举了c标准库中的一些（不是全部）线程不安全函数，涉及多线程或者其它并发情况，谨慎使用。

STRTok(3)

Linux Programmer's Manual

STRTok(3)

## NAME

`strtok, strtok_r - extract tokens from strings`

## SYNOPSIS

`#include <string.h>``char *strtok(char *str, const char *delim);``char *strtok_r(char *str, const char *delim, char **saveptr);`

## DESCRIPTION

`...`

The `strtok_r()` function is a reentrant version `strtok()`. The `saveptr` argument is a pointer to a `char *` variable that is used internally by `strtok_r()` in order to maintain context between successive calls that parse the same string.

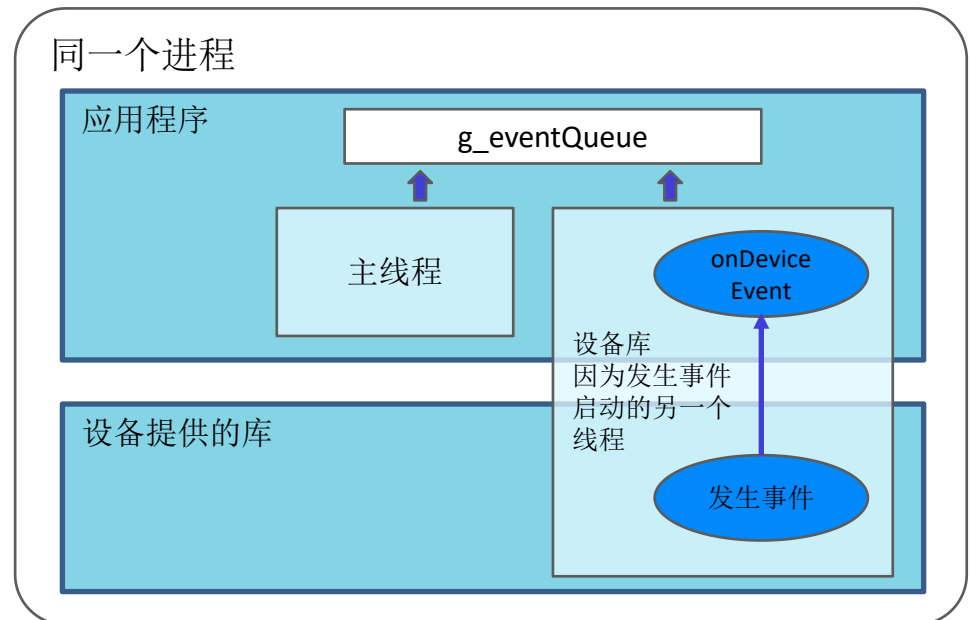
`...`

一般来说，c标准库中所有线程不安全函数，都会对应有一个线程安全的版本（上图中是`strtok`的man页面），可以在需要的时候使用线程安全版本。

```
// OnDeviceEvent是某设备SDK定义的一个回调函数，当设备有事件发生的时候会调用它
void OnDeviceEvent(int eventId, const char* para, int paraLength)
{
    g_eventQueue.add(eventId, para, paraLength); // 将事件加入到一个全局的事件队列
}
```

```
// OnDeviceEvent是某设备SDK定义的一个回调函数，当设备有事件发生的时候会调用它
// 这个回调函数设备的SDK库中启动了一个单独的线程调用的
void OnDeviceEvent(int eventId, const char* para, int paraLength)
{
    LOCK(g_eventQueue) // 用pthread_mutex_lock实现的一个宏定义，资源加锁
    g_eventQueue.add(eventId, para, paraLength); // 将事件加入到一个全局的事件队列
    UNLOCK(g_eventQueue) // 用pthread_mutex_unlock实现的一个宏定义，资源解锁
}
```

在基于某些设备的SDK编程的时候，实现回调函数时要特别注意：这个回调函数是否是另一个线程调用的？



```
struct A
{
    char* p;
    A() { p = NULL; }
    ~A() { if(p != NULL) delete p; }
    void init(int size) { p = new char[size]; }
};
```

```
A* a = new A[10];
a[0].init(100);
a[1].init(200);
a[2].init(400);
...
delete a;
```

```
A* a = new A[10];
a[0].init(100);
a[1].init(200);
a[2].init(400);
...
delete []a;
```

含有析构函数的对象数组，删除的时候不要忘记使用 **delete[]**，否则会导致内存泄露或者程序崩溃。

```
struct A
{
    int i;
    ~A() {}
};
struct B : public A
{
    char* p;
    B(int size) { p = new char[size]; }
    ~B() { delete p; }
}

...
A* a = new B(100);
...
delete a; // 这里只有~A()会被调用
```

```
struct A
{
    int i;
    virtual ~A() {}
};
struct B : public A
{
    char* p;
    B(int size) { p = new char[size]; }
    ~B() { delete p; }
}

...
A* a = new B(100);
...
delete a; // ~A()和~B()都会被调用
```

不要忘记将基类析构函数声明为虚函数，否则会造成内存泄漏

# Sample Class: Points

---

```
class Pt {  
    public:  
        Pt(int xv);  
        Pt(Pt* pv);  
        int getX();  
        virtual void move(int dx);  
    protected:  
        void setX(int xv);  
    private:  
        int x;  
};
```

Overloaded constructor

Public read access to private data

Virtual function

Protected write access

Private member data



# Virtual Functions

---

- **Virtual** member functions
  - Accessed by indirection through pointer in object
  - May be redefined in derived subclasses
  - The exact function to call determined dynamically
- Non-virtual functions are ordinary functions
  - Cannot redefine in subclasses (but can overload)
- Member functions are virtual if explicitly declared or inherited as virtual, else non-virtual

# Sample Derived Class: Color Point

---

```
class ColorPt: public Pt {  
    public:
```

Public base class gives supertype

```
        ColorPt(int xv,int cv);  
        ColorPt(Pt* pv,int cv);  
        ColorPt(ColorPt* cp);
```

} Overloaded constructor

```
        int getColor();
```

Non-virtual function

```
        virtual void move(int dx);  
        virtual void darken(int tint);
```

} Virtual functions

```
    protected:
```

```
        void setColor(int cv);
```

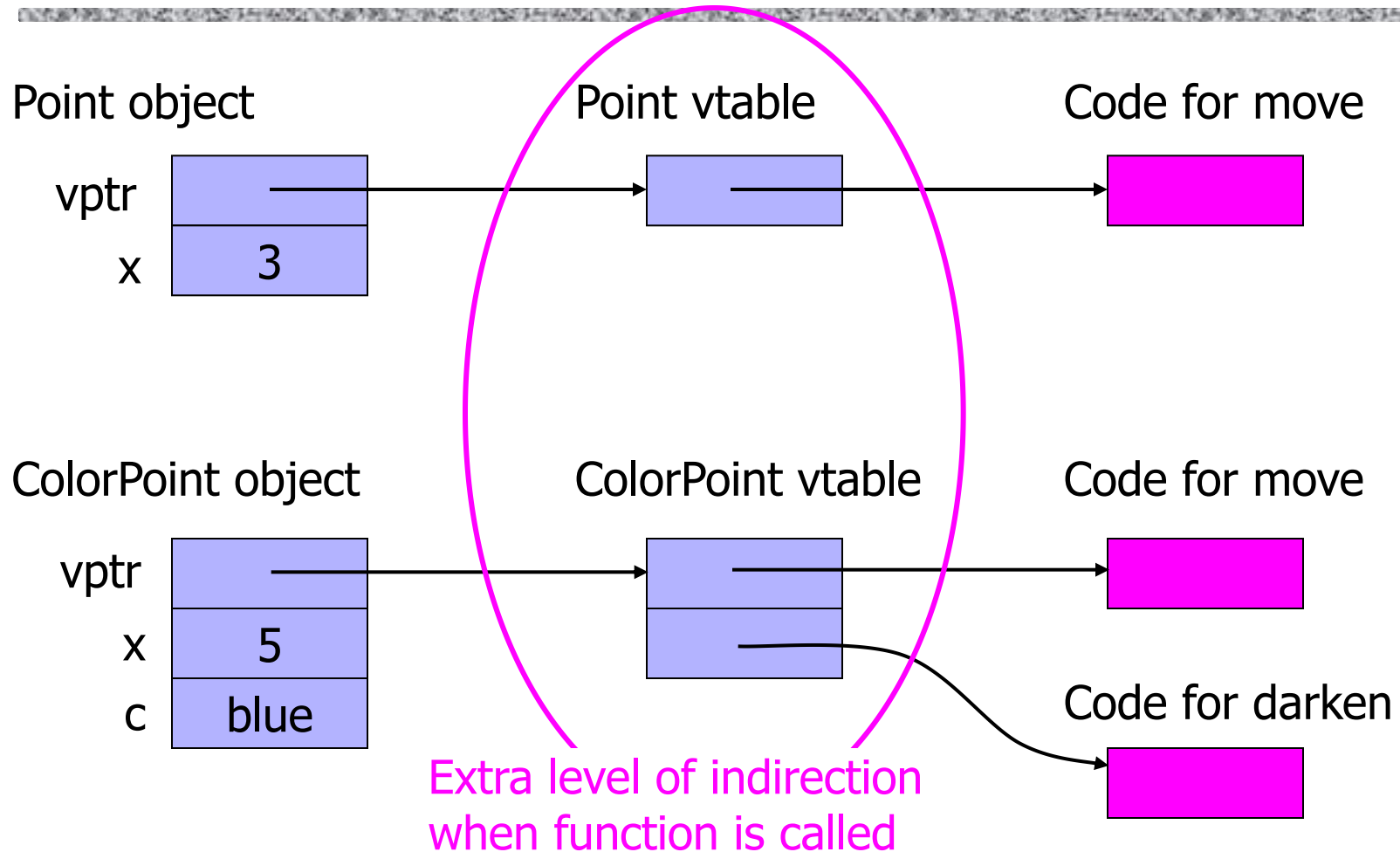
Protected write access

```
    private:
```

```
        int color; };
```

Private member data

# Run-Time Representation



Data at same offset

Function pointers at same offset

```
struct A
{
    virtual void foo() {}
    int serialize(char* buffer, int bufSize);
    bool deserialize(char* buffer, int size);
    int size();
    int i;
    int j;
};
```

```
A a1;
a1.i = 100;
a1.j = 200;
char buffer[MAX_SIZE];
int size = a1.serialize(buffer, MAX_SIZE);
if(size == 0)
    // 出错处理
else
    // 对buffer的前size字节进行传输或者存储
```

```
// 另一个程序获取到buffer和size
A a2;
bool succ = a2.deserialize(buffer, size);
```

含有虚函数的对象的序列化，  
需要注意“虚表指针”的问题。

```
int A::size() { return sizeof(A); }
int A::serialize(char* buffer, int bufSize)
{
    if(bufSize < size())
        return 0;
    memcpy(buffer, (char*)this, size());
    return size();
}
bool A::deserialize(char* buffer, int bufSize)
{
    if(bufSize != size())
        return false;
    memcpy((char*)this, buffer, size());
    return true;
}
```

```
int A::size() { return sizeof(i) + sizeof(j); }
int A::serialize(char* buffer, int bufSize)
{
    if(bufSize < size())
        return 0;
    memcpy(buffer, (char*)&i, sizeof(i));
    buffer += sizeof(i);
    memcpy(buffer, (char*)&j, sizeof(j));
    return size();
}
bool A::deserialize(char* buffer, int bufSize)
{
    if(bufSize != size())
        return false;
    memcpy((char*)&i, buffer, sizeof(i));
    buffer += sizeof(i);
    memcpy((char*)&j, buffer, sizeof(j));
    return true;
}
```

# 小实验作业“程序排错”

本周四<http://10.112.112.240:8081/2010813434/ppt>下的“小作业2”中会发布一个存在BUG的程序kmp\_bad.c。

这个程序是针对字符串匹配的kmp算法执行过程的一个演示（最好在Linux环境下运行，Windows环境下也可以，但是由于终端功能局限效果不太好）。

要求如下：

- 将程序编译，运行，给出BUG现象说明。
- 找出其中的两个BUG，给出查找的过程和依据。
- 对两个BUG的位置，BUG产生的原因，修改方法做出描述。
- 修改两个BUG，修改后的程序能够正确运行。

提交内容：

- 一个纯文本文件，针对上述要求给出文字说明。
- 一个修改后可以运行正确的程序kmp.c。

截止时间：

- 作业发布之后一周内，将上述两个文件提交到自己的GitLab账号所建的小作业2项目中。